IT5005 Project: Vision–Language Models with CLIP

AY 2026/27 Semester 1

Instructions

- **Groups:** all members are expected to contribute to the code and to the video presentation.
- **Submission:** through the course Canvas site. See Section 3 for the deliverables and file names.
- **Due date:** Tuesday, 17 November 2026, for all components.
- **Where to answer:** all written answers go in the *"Your answer"* Markdown cells of the notebooks. There is **no separate PDF report**.
- **Generative AI policy:** you may use any generative AI tool, but you must document its use in the *AI Usage* cell at the end of each notebook (which tool, what you asked it, and where you used it). You are responsible for the correctness of everything you submit. Because code and written answers can be produced with AI help, the **video presentation** (Section 2) is where you show that *you* understand the work.
- **Late policy:** no late submissions are accepted.

Component	Content	Marks
Part 1 notebook	Zero-shot classification and probing on ImageNet	20
Part 2 notebook	Projection-only and LoRA fine-tuning for image–text retrieval on Flickr8k	20
Video presentation	30 minutes covering **both** parts	60
Total		**100**

The tasks, the mark of each step and the written questions are given in the notebooks.

1 Project Overview

Early AI models were trained on a single modality, e.g. images only or text only. A single modality often lacks the expressive power to convey complicated ideas (imagine describing what an alien looks like to a vision-impaired friend). Different modalities complement each other and better reflect how humans experience the world, so multi-modal models, especially vision–language models, have made rapid progress.

This project explores CLIP [1], which learns a shared embedding space for images and text with a contrastive objective. It has two parts:

- **Part 1** (`part_1.ipynb`): CLIP as a *frozen* feature extractor. You implement zero-shot image classification on ImageNet, study the effect of prompts and of the similarity measure, train a small classification head (a probe) on frozen CLIP features, and answer questions on the CLIP paper. The data are split into train / validation / test sets: the prompt and

the training epoch are chosen on validation (with early stopping), and the test set is used only for the final evaluation of both methods.

- **Part 2** (`part_2.ipynb`): *adapting* CLIP to image–text retrieval on Flickr8k with two parameter-efficient fine-tuning (PEFT) methods: training only the projection layers, and LoRA [2]. You implement the InfoNCE [3] and triplet [4] contrastive losses, Recall@K, a LoRA layer from scratch, and the training step. As in Part 1, the learning rate, epoch and LoRA rank are chosen on a validation set, training uses early stopping, and the test set is used only for the final evaluation of the selected model of each method and, after that, for illustrative retrieval examples (one success and one failure).

Frozen encoder outputs are computed once and cached to disk, so later runs are fast. The first run of Part 1 encodes 50,000 ImageNet images, which can take an hour or more on a CPU. Both notebooks must run from top to bottom without errors; non-runnable notebooks are penalised.

2 Video Presentation (both parts)

Each group records **one video of 30 minutes** covering Parts 1 and 2. Its purpose is to show that every member understands the CLIP architecture, the loss functions, the fine-tuning strategies, and the limitations of these methods, using *your own* results as evidence.

2.1 Format

- Length: **30 minutes**. Content beyond 30 minutes is not marked.
- Screen recording (slides and/or your notebooks) with voice-over and webcam.
- **Every member presents** at least one of the checkpoint questions below, with roughly equal speaking time. State your name when you start speaking.
- Every answer must refer to your group's own outputs: a printed result, a plot, or a table from your notebooks. Reading out notebook text or giving generic explanations scores low.

2.2 Checkpoint questions

Start with a short introduction (group members, project goal, outline of the video), then answer all twelve questions in this order.

#	Checkpoint question
	Part 1: CLIP and zero-shot learning
1	**Architecture.** Trace an image and a caption through CLIP ViT-B/32 to the similarity score: patches and tokens, the two encoders, which output token is used on each side, the projection layers, normalisation and `logit_scale`. Why are the projection layers needed?
2	**Zero-shot classifier.** Walk through your `encode_image`, `encode_text` and `predict`. How do 1,000 class names become a classifier without training?
3	**Similarity measure.** Did cosine similarity and the dot product give different results in your notebook? Show your numbers. When do the two measures rank classes identically, and what changes when the embeddings are not L2-normalised: which vector norms can change the predicted class, and which cannot?
4	**Prompts.** Which prompts worked best, are the differences meaningful, and why does an unrelated prefix hurt? What role does the 77-token limit play?

#	Checkpoint question
5	**Probe vs. zero-shot.** Compare their test accuracies (probe at the epoch chosen on validation). Why does the trained head outperform zero-shot, and what is lost in exchange (data, flexibility, vocabulary)?
	Part 2: Losses and fine-tuning
6	**InfoNCE.** Explain the loss on a batch of four image–caption pairs: positives, negatives, labels, both directions, and the role of the temperature.
7	**InfoNCE vs. triplet.** Compare their curves and test recall, and explain the difference through the negatives each loss uses.
8	**Fine-tuning strategies.** Compare the Part 1 probe, projection-only PEFT and LoRA: what each one trains, how many parameters, and what each can and cannot change.
9	**LoRA.** Give the LoRA equation and the shapes of A and B; derive the parameter count for $r = 8$; explain the $B = 0$ initialisation, merging, and your rank-sweep result.
10	**Methodology.** Why do both parts use a separate validation set, and what did you choose with it? Read your learning-rate and 25-epoch plots: where are under-fitting and overfitting, and when would early stopping with `patience=3` have stopped training?
11	**Results.** Present your final test table for Part 2 and state the most important finding. How confident are you in the differences between methods, given the test-set size and run-to-run variation?
	Wrap-up
12	**Limitations.** Where does CLIP fall short (with an example from your results)? What can projection-only and LoRA not do? What might fine-tuning on Flickr8k have made worse, and how would you test it?

3 Deliverables

Submit the following on Canvas by Tuesday, 17 November 2026.

Item	File name
Part 1 notebook, run top to bottom with all outputs saved, including the *AI Usage* cell	`group_<id>_part_1.ipynb`
Part 2 notebook, run top to bottom with all outputs saved, including the *AI Usage* cell	`group_<id>_part_2.ipynb`
Video presentation covering both parts (MP4), or a text file with a viewable link to it	`group_<id>_video.mp4` or `group_<id>_video.txt`

Before submitting, restart the kernel and run all cells (*Kernel* → *Restart & Run All*), then save. Do not submit the `cache` folders.

4 Environment Setup

4.1 Install Conda

We use `conda` to isolate this project's packages from others you may have installed and to minimise version conflicts. We recommend `miniconda` (guide here); `anaconda` with a graphical interface also works (guide here). Getting familiar with terminal commands will benefit you in the future.

4.2 Create the Environment

```
conda create -n it5005project python=3.12 -y
conda activate it5005project
conda install pip -y
```

4.3 Install the Packages

The required packages are listed in `project_requirements.txt`:

```
pip install -r project_requirements.txt
```

The notebooks require `torch>=2.4`, `transformers>=5.0` (4.4x also works), `datasets>=3.0`, `scikit-learn>=1.4`, `matplotlib`, `tensorboard`, `tqdm`, `pillow` and `requests`; `peft` is optional (Part 2). Installation may take some time. Part 1 also needs the file `id_to_label.json` (provided) in the same folder as the notebook. Training curves can be viewed with `tensorboard --logdir runs`.

References

[1] A. Radford, J. W. Kim, C. Hallacy, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. https://arxiv.org/abs/2103.00020

[2] E. J. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022. https://arxiv.org/abs/2106.09685

[3] A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. *arXiv:1807.03748*, 2018. https://arxiv.org/abs/1807.03748

[4] F. Faghri, D. J. Fleet, J. R. Kiros, and S. Fidler. VSE++: Improving visual-semantic embeddings with hard negatives. In *BMVC*, 2018. https://arxiv.org/abs/1707.05612